

---

# Scalable Automatic Differentiation of Hyperbolic Neural Networks

---

**Tiberiu Iancu**  
University of Amsterdam

**Pascal Mettes**  
VIS Lab, Informatics Institute  
University of Amsterdam

**Max van Spengler**  
VIS Lab, Informatics Institute  
University of Amsterdam

## Abstract

Hyperbolic neural networks are effective at modeling hierarchical and graph-structured data but often suffer from poor GPU utilization. Existing implementations typically decompose hyperbolic layers into long chains of element-wise operations, resulting in memory-bound computation and limiting scalability. In this work, we present a fused GPU implementation of the Poincaré fully connected layer that addresses these inefficiencies through Triton-based kernel fusion. By collapsing intermediate operations, reducing memory traffic, and producing a compact autograd graph, our approach maintains full mathematical correctness while achieving substantial performance gains. Experiments on standard benchmarks demonstrate up to 2× speedup and approximately 50% reduction in activation memory compared to the reference PyTorch implementation, significantly narrowing the efficiency gap between hyperbolic and Euclidean networks. This work provides a foundation for geometry-aware acceleration in deep learning on curved spaces. The implementation is available at [github.com/tiberuiancu/hyperbolic\\_learning\\_library](https://github.com/tiberuiancu/hyperbolic_learning_library).

## 1 Introduction

Hyperbolic representations are now practical ways of learning hierarchies, with the Poincaré ball being the most common of the hyperbolic models. As libraries like HypLL help move hyperbolic networks from isolated layers to full architectures (e.g., Poincaré ResNet [13]), training and inference still fail to scale effectively, constrained by long operator chains and heavy memory traffic.

Because hyperbolic operations are computationally intensive and hardware is tuned for Euclidean operations, current implementations materialize large intermediate tensors across many small operations—making memory bandwidth, rather than compute, the primary bottleneck. This arises from the long chains of element-wise functions typical in hyperbolic models: each operation reads and writes to global memory separately, causing excessive data movement and poor cache utilization. Without fused kernels to keep intermediate results in registers or shared memory, GPUs spend far more time on memory transfers than on arithmetic, leading to inefficient execution despite ample compute capacity.

In this work, we take the first step toward geometry-aware acceleration by focusing on the simplest building block: a fully connected layer with ReLU activation. The goal is to establish a path for hyperbolic layer optimization. Following the design philosophy of FlashAttention [4] in transformers, we fuse the forward and backward passes into custom GPU kernels built on HypLL [14], while preserving the Poincaré-ball formulation of the layer introduced in HNN++ [12]. The result is a

compact autograd graph with fewer intermediates and a reduced memory footprint, provided as a drop-in replacement.

Our main contributions are as follows: first, we provide an analysis of the computational structure of the Poincaré fully connected layer, highlighting sources of inefficiency in existing implementations. Second, we introduce a fused Triton implementation of the Poincaré-ball feed-forward layer with activation and a corresponding custom backward pass, integrated as a drop-in replacement within HypLL. For completeness, we also derive and include the manual gradients. Third, we present empirical evidence—showing up to  $2\times$  speedups and reduced memory usage—that kernel fusion can substantially narrow the efficiency gap between hyperbolic and Euclidean networks without compromising model quality.

We release our code publicly as a fork of HypLL, available at [github.com/tiberuiiancu/hyperbolic\\_learning\\_library](https://github.com/tiberuiiancu/hyperbolic_learning_library).

## 2 Background and Related Work

### 2.1 Hyperbolic Neural Networks

Hyperbolic geometry—most commonly the Poincaré ball model—naturally accommodates hierarchical data thanks to its exponential volume growth, enabling low-distortion embeddings of tree-like structures that challenge Euclidean spaces [8].

Early work often adopted a “hyperbolic-at-the-end” setup, where Euclidean feature extractors fed into a hyperbolic embedding or classification layer [8]. This evolved with the introduction of Hyperbolic Neural Networks (HNNs), which redefined standard components using Möbius gyrovector calculus to operate directly on the manifold, producing hyperbolic versions of linear layers, logistic classifiers, and recurrent models [5]. HNN++ [12] expanded this framework by introducing geodesic-aware analogues of fully connected, convolutional, and attention layers in the Poincaré ball, achieving comparable or better performance with fewer parameters than their Euclidean counterparts [12]. The latest step toward fully curved architectures comes from Poincaré ResNets, which compute every layer—convolutions, residual connections, and activations—within the manifold, realizing true end-to-end hyperbolic learning [13].

In parallel, toolkits (e.g., HypLL) have consolidated manifold operations and hyperbolic layers into open-source libraries, lowering the barrier to composing full hyperbolic architectures and reproducing published models [14]. These developments establish a mature toolkit for deep learning on curved spaces.

### 2.2 GPU Kernel Fusion

A GPU’s memory is organized in a performance hierarchy, where each layer offers a different trade-off between speed and capacity. The fastest tiers—registers and on-chip shared memory—offer low-latency access and can feed large numbers of concurrent threads. High-bandwidth memory (HBM) provides much greater capacity but at far higher access cost.

Modern Euclidean deep learning frameworks (e.g., PyTorch [11]) leverage GPU kernels explicitly designed for the memory hierarchy. These kernels, typically sourced from vendor libraries like cuBLAS [3] and cuDNN [2], manage thread coordination and data movement to retain intermediate values in registers or shared memory, avoiding costly spills to HBM. To further minimize HBM traffic, a key optimization is kernel fusion: combining operations like matrix multiplication and activation into a single kernel to preserve on-chip data reuse. While such optimizations previously required low-level CUDA [9] expertise, domain specific languages (DSLs) like Triton [10] now provide high-level abstractions for generating hardware-aware kernels.

In contrast, hyperbolic neural network layers are typically implemented as a long sequence of small, element-wise operations (e.g., Möbius addition, exponential and logarithmic maps) where each operation is launched as a separate GPU kernel. This design forces each intermediate result to be materialized to HBM, only to be immediately reloaded by the subsequent kernel. The resulting cascade of small, memory-bound kernels underutilizes the GPU’s compute capabilities, as the system spends more time moving data than performing arithmetic.

This problem of memory-bound operations has been successfully addressed in other domains through the design of custom, IO-aware kernels. A seminal example is FlashAttention [4], which re-implements the standard attention mechanism [1] to avoid materializing the large intermediate attention matrix in HBM. By tiling the inputs to fit in SRAM and recomputing parts of the calculation on-chip, it achieves significant speedups and memory savings. We propose a similar solution for hyperbolic geometry: by fusing the sequence of elemental hyperbolic operations into custom kernels, we can minimize HBM traffic and shift the performance bottleneck from memory to compute.

### 2.3 Poincaré Fully Connected Layer Formulation

We focus on the fully connected (FC) layer in the Poincaré ball, following the formulation introduced in HNN++ [12]. This section provides an overview of the layer’s mathematical structure.

Given an input  $\mathbf{x} \in \mathbb{B}_c^n$  on a Poincaré ball of curvature  $-c$ , the hyperbolic fully connected (FC) layer is defined as:

$$\mathbf{y} = F_c(\mathbf{x}; \mathbf{Z}, \mathbf{r}) = \frac{\mathbf{w}}{1 + \sqrt{1 + c\|\mathbf{w}\|^2}}, \quad \mathbf{w} = \left( c^{-\frac{1}{2}} \sinh(\sqrt{c} v_k(\mathbf{x})) \right)_{k=1}^m. \quad (1)$$

Here,  $\mathbf{Z} = \{\mathbf{z}_k \in T_0\mathbb{B}_c^n = \mathbb{R}^n\}_{k=1}^m$  are the learnable weight vectors,  $\mathbf{r} = \{r_k \in \mathbb{R}\}_{k=1}^m$  are bias terms, and  $v_k(\mathbf{x})$  is the hyperbolic analogue of the Euclidean pre-activation  $\langle \mathbf{z}_k, \mathbf{x} \rangle + r_k$ , and is defined as:

$$v_k(\mathbf{x}) = \frac{2}{\sqrt{c}\|\mathbf{z}_k\|} \sinh^{-1} \left( \lambda_{\mathbf{x}}^c \langle \sqrt{c}\mathbf{x}, \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|} \rangle \cosh(2\sqrt{c}r_k) - (\lambda_{\mathbf{x}}^c - 1) \sinh(2\sqrt{c}r_k) \right) \quad (2)$$

where  $\lambda_{\mathbf{x}}^c := \frac{2}{1-c\|\mathbf{x}\|^2}$  represents the conformal factor of the Poincaré ball model, which rescales Euclidean quantities to account for hyperbolic geometry. In the flat limit  $c \rightarrow 0$ , this formulation converges to the standard Euclidean linear layer.

After the Poincaré fully connected layer, we apply a projection step to ensure that the resulting embeddings remain within the valid hyperbolic space. Given the curvature parameter  $c > 0$ , we project each output vector  $\mathbf{y}$  to satisfy the constraint  $\|\mathbf{y}\| < 1/\sqrt{c}$ . Specifically, we define

$$\tilde{\mathbf{y}} = \begin{cases} \mathbf{y}, & \text{if } \|\mathbf{y}\| \leq \frac{1-\epsilon}{\sqrt{c}}, \\ \frac{\mathbf{y}}{\|\mathbf{y}\|} \cdot \frac{1-\epsilon}{\sqrt{c}}, & \text{otherwise,} \end{cases} \quad (3)$$

where  $\epsilon$  is a small positive constant (e.g.,  $4 \times 10^{-3}$  for float32) ensuring that points remain strictly inside the Poincaré ball. This operation prevents numerical instability and guarantees that the embeddings lie on a valid manifold.

Since standard nonlinearities like ReLU are not directly defined on curved manifolds, we apply them in the tangent (Euclidean) space at the origin.

$$\tilde{\mathbf{y}} \xrightarrow{\text{logmap}_0} \mathbf{v} \xrightarrow{\text{ReLU}} \mathbf{v}' \xrightarrow{\text{expmap}_0} \mathbf{y}', \quad (4)$$

where  $\text{logmap}_0$  and  $\text{expmap}_0$  move between the Poincaré ball and its Euclidean tangent space at the origin and are defined as:

$$\text{log}_0^c(\tilde{\mathbf{y}}) = \text{artanh}(\sqrt{c}\|\tilde{\mathbf{y}}\|) \frac{\tilde{\mathbf{y}}}{\sqrt{c}\|\tilde{\mathbf{y}}\|}, \quad \text{exp}_0^c(\mathbf{v}) = \tanh(\sqrt{c}\|\mathbf{v}\|) \frac{\mathbf{v}}{\sqrt{c}\|\mathbf{v}\|}. \quad (5)$$

Here  $c > 0$  is the curvature parameter and  $\|\cdot\|$  denotes the Euclidean norm. The exponential map  $\text{exp}_0^c$  projects a tangent vector  $v$  from the Euclidean tangent space  $T_0\mathbb{B}_c^n$  onto the hyperbolic manifold  $\mathbb{B}_c^n$ , while the logarithmic map  $\text{log}_0^c$  performs the inverse operation, mapping a point on the manifold back to the tangent space.

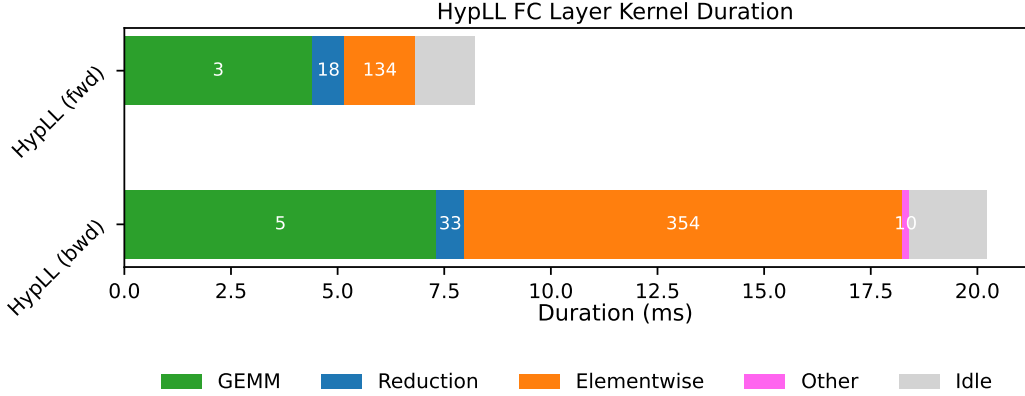


Figure 1: GPU kernel executions per training step of single layer perceptron, grouped by kernel type: per-element execution, reductions across rows or columns, and general matrix multiplications (GEMM). Each region shows the number of GPU kernels executed. Idle is the time that the GPU remains unutilized.

### 3 Fused Poincaré Fully Connected Layer

This section describes our approach to making the Poincaré FC layer GPU-efficient. We first profile the existing HypLL implementation to identify bottlenecks, then present our optimization: (i) decompose the layer into atomic ops and derive the backward pass, (ii) fuse the elementwise work into a small number of Triton kernels while delegating large GEMMs to cuBLAS, and (iii) handle the tangent-space ReLU in a modular, fused fashion. Appendix A contain additional implementation details.

#### 3.1 Profiling the HypLL implementation

We profile a single-layer hyperbolic perceptron (input = output = 4096, batch = 256) trained on data generated randomly for one epoch and record GPU traces with PyTorch’s profiler. Figure 1 visualizes the result: the forward and backward passes are dominated by many small element-wise kernels, and the GPU spends a significant fraction of time idle (roughly 10%). The behavior indicates a memory and launch bound workload: numerous independent kernels force frequent reads/writes to HBM.

The results indicate that the most effective optimization strategy is to fuse the element-wise operations that dominate execution time. Reducing the number of separate kernels is particularly important when using Triton, as it introduces additional launch overhead compared to CUDA; our approach should therefore compute element-wise operations and reductions in the same kernel. The matrix multiplications themselves are already handled by highly optimized cuBLAS routines and are therefore not a source of inefficiency.

#### 3.2 Forward Pass Kernel

Our forward pass implementation follows the formulation presented in section 2.3. Here we introduce the kernel strategy and optimization tricks.

Our task is to write a kernel that receives as inputs the 2D batched input matrix  $\mathbf{X}$  (where each row represents an input sample  $\mathbf{x}$ ), weight matrix  $\mathbf{Z}$ , and biases  $\mathbf{r}$ , and produces the post-activation outputs  $\mathbf{y}'$ .

We first note that the forward computation leverages two compute-intensive operations: Euclidean weight norms  $\|\mathbf{z}_k\|$  and a matrix multiplication  $\mathbf{XZ}$  within the computation of each  $v_k(\mathbf{x})$ . We compute these via PyTorch’s optimized routines. The remaining hyperbolic operations are fused into a single Triton kernel with a program launch grid sized to the batch dimension, enabling each program to process one batch element. This design allows efficient reduction operations within the same kernel through multiple passes over the input features.

Thus our kernel implements 3 passes over the input: first, each program computes  $\lambda_{\mathbf{x}}^c$  for its respective batch input  $\mathbf{x}$ , by iterating over the input dimension. A second pass over the output dimension  $\mathbf{M}$  computes the nominator  $\mathbf{w}$  and accumulates the denominator  $1 + \sqrt{1 + c\|\mathbf{w}\|^2}$ . Finally, a last pass over the output dimension  $\mathbf{M}$  computes the output  $\mathbf{y}$  and the projection  $\tilde{\mathbf{y}}$ . Note that the computation of  $\mathbf{y}$  is only needed for the backward derivation.

For ease of development and derivation, we implement the activation in a separate Triton kernel that takes as input the projected outputs  $\tilde{\mathbf{y}}$ , and maps it to the tangent space at origin via  $\text{logmap}_0$ , performs ReLU, and maps it back to the hyperbolic manifold via  $\text{expmap}_0$ . We fuse this in one kernel via a two pass approach: first, we compute the norms  $\|\tilde{\mathbf{y}}\|$  and  $\|\mathbf{v}\|$  by iterating over the output dimension  $M$ , then in a second pass calculate the output by performing the element-wise operations, as detailed in section 2.3.

### 3.3 Backward Pass Formulation

Custom kernels in automatic differentiation engines require manual derivation and implementation of a backward pass. Thus in this section we detail the derivation, and implementation details of our backward kernels.

Given  $\nabla_{\mathbf{v}'}\mathcal{L}$  – the derivative of the loss function w.r.t. the layer outputs  $\mathbf{v}'$  – we calculate  $\nabla_{\mathbf{x}}\mathcal{L}$ ,  $\nabla_{\mathbf{z}}\mathcal{L}$ , and  $\nabla_{\mathbf{r}}\mathcal{L}$ , the derivatives of the loss with respect to the batched input matrix, layer weights, and layer biases respectively, by propagating backwards through the operations chain:

**Activation  $\tilde{\mathbf{y}} \mapsto \mathbf{y}'$ .**

$$\nabla_{\mathbf{v}'}\mathcal{L} = \frac{\tanh(\sqrt{c}\|\mathbf{v}'\|)}{\sqrt{c}\|\mathbf{v}'\|} \nabla_{\mathbf{y}'}\mathcal{L} + \frac{\sqrt{c}\|\mathbf{v}'\| \operatorname{sech}^2(\sqrt{c}\|\mathbf{v}'\|) - \tanh(\sqrt{c}\|\mathbf{v}'\|)}{c\|\mathbf{v}'\|^3} (\mathbf{v}'^\top \nabla_{\mathbf{y}'}\mathcal{L}) \mathbf{v}'. \quad (6)$$

$$\nabla_{\tilde{\mathbf{y}}}\mathcal{L} = \mathbf{1}_{\mathbf{v}>0} \odot \left[ \frac{\tanh^{-1}(\sqrt{c}\|\tilde{\mathbf{y}}\|)}{\sqrt{c}\|\tilde{\mathbf{y}}\|} \nabla_{\mathbf{v}'}\mathcal{L} + \frac{\frac{\sqrt{c}\|\tilde{\mathbf{y}}\|}{1-(\sqrt{c}\|\tilde{\mathbf{y}}\|)^2} - \tanh^{-1}(\sqrt{c}\|\tilde{\mathbf{y}}\|)}{\|\tilde{\mathbf{y}}\|^3} (\tilde{\mathbf{y}}^\top \nabla_{\mathbf{v}'}\mathcal{L}) \tilde{\mathbf{y}} \right], \quad (7)$$

or  $\nabla_{\tilde{\mathbf{y}}}\mathcal{L} = \nabla_{\mathbf{y}'}\mathcal{L}$  without activation.

**Gradient with respect to input  $\mathbf{x}$ .** For arbitrary indices  $b, i$ :

$$\frac{\partial \mathcal{L}}{\partial x_{bi}} = \sum_k \nabla_{\tilde{y}_{bk}}\mathcal{L} \frac{\partial \tilde{y}_{bk}}{\partial x_{bi}} = \sum_k \underbrace{\left( \sum_j \nabla_{\tilde{y}_{bj}}\mathcal{L} \frac{\partial \tilde{y}_{bj}}{\partial y_{bk}} \right)}_{=\nabla_{y_{bk}}\mathcal{L}} \frac{\partial y_{bk}}{\partial x_{bi}} \quad (8)$$

The gradient through projection is:

$$\nabla_{y_{bk}}\mathcal{L} = \begin{cases} \nabla_{\tilde{y}_{bk}}\mathcal{L}, & \text{if } \|\mathbf{y}_b\| \leq \frac{1-\epsilon}{\sqrt{c}}, \\ \frac{c}{\|\mathbf{y}_b\|} \left( \nabla_{\tilde{y}_{bk}}\mathcal{L} - \|\mathbf{y}_b\|^{-2} y_{bk} \sum_j \nabla_{\tilde{y}_{bj}}\mathcal{L} y_{bj} \right), & \text{if } \|\mathbf{y}_b\| > \frac{1-\epsilon}{\sqrt{c}}. \end{cases}$$

Since  $\mathbf{y}_b = \frac{\mathbf{w}_b}{1 + \sqrt{1 + c\|\mathbf{w}_b\|^2}}$ :

$$\frac{\partial y_{bk}}{\partial x_{bi}} = \frac{1}{d_b} \frac{\partial w_{bk}}{\partial x_{bi}} - \frac{w_{bk}}{d_b^2} \frac{c}{d_b - 1} \sum_j w_{bj} \frac{\partial w_{bj}}{\partial x_{bi}}, \quad (9)$$

where  $d_b := 1 + \sqrt{1 + c\|\mathbf{w}_b\|^2}$ . Thus:

$$\frac{\partial \mathcal{L}}{\partial x_{bi}} = \sum_k \frac{1}{d_b} \underbrace{\left( \nabla_{y_{bk}}\mathcal{L} - \frac{c w_{bk}}{d_b(d_b - 1)} \sum_j \nabla_{y_{bj}}\mathcal{L} w_{bj} \right)}_{:=\nabla_{w_{bk}}\mathcal{L}} \frac{\partial w_{bk}}{\partial x_{bi}} \quad (10)$$

Backwards through  $w_{bk} = c^{-1/2} \sinh(\sqrt{c} v_k(\mathbf{x}_b))$ :

$$\frac{\partial w_{bk}}{\partial x_{bi}} = \cosh(\sqrt{c} v_k(\mathbf{x}_b)) \frac{\partial v_k(\mathbf{x}_b)}{\partial x_{bi}}, \quad (11)$$

where the gradient of  $v_k$  with respect to  $\mathbf{x}$  is:

$$\frac{\partial v_k(\mathbf{x}_b)}{\partial x_{bi}} = \frac{2}{\sqrt{c} \|\mathbf{z}_k\|} \frac{1}{\sqrt{1 + a_{bk}^2}} \frac{\partial a_{bk}}{\partial x_{bi}}, \quad (12)$$

where

$$a_{bk} := \lambda_b \langle \sqrt{c} \mathbf{x}_b, \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|} \rangle \cosh(2\sqrt{c} r_k) - (\lambda_b - 1) \sinh(2\sqrt{c} r_k), \quad (13)$$

and the derivative of  $a_{bk}$  is:

$$\frac{\partial a_{bk}}{\partial x_{bi}} = \lambda_b \cosh(2\sqrt{c} r_k) \sqrt{c} \frac{z_{ki}}{\|\mathbf{z}_k\|} + c \lambda_b^2 x_{bi} \left( \langle \sqrt{c} \mathbf{x}_b, \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|} \rangle \cosh(2\sqrt{c} r_k) - \sinh(2\sqrt{c} r_k) \right). \quad (14)$$

Combining terms, we get the following expression in index notation:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x_{bi}} &= x_{bi} \sum_k c \lambda_b^2 \nabla_{w_{bk}} \mathcal{L} \cosh(\sqrt{c} v_k(\mathbf{x}_b)) \left( \langle \sqrt{c} \mathbf{x}_b, \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|} \rangle \cosh(2\sqrt{c} r_k) - \sinh(2\sqrt{c} r_k) \right) \\ &\quad + \sum_k \nabla_{w_{bk}} \mathcal{L} \cosh(\sqrt{c} v_k(\mathbf{x}_b)) \lambda_b \cosh(2\sqrt{c} r_k) \sqrt{c} \frac{z_{ki}}{\|\mathbf{z}_k\|}, \end{aligned} \quad (15)$$

which can be re-written in vectorized form:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \mathbf{X} \text{diag}(\mathbf{t}^{(x)}) + \mathbf{S}^{(x)} \mathbf{Z}_{\text{norm}}, \quad (16)$$

where

$$\mathbf{t}^{(x)} := \left( c \lambda \lambda^\top \odot (\nabla_{\mathbf{W}} \mathcal{L} \odot \cosh(\sqrt{c} \mathbf{V})) \mathbf{M}^\top \right), \quad (17)$$

$$\mathbf{S}^{(x)} := (\nabla_{\mathbf{W}} \mathcal{L} \odot \cosh(\sqrt{c} \mathbf{V}) \odot (\lambda \mathbf{c}^\top)). \quad (18)$$

Here, we employ the following notation:

$$\begin{aligned} \mathbf{M} &= [\mathbf{m}_1, \dots, \mathbf{m}_m]^\top \in \mathbb{R}^{m \times 2} \quad \text{where} \quad \mathbf{m}_k = (\cosh(2\sqrt{c} r_k), -\sinh(2\sqrt{c} r_k)), \\ \lambda &= (\lambda_1, \dots, \lambda_B)^\top \in \mathbb{R}^B, \\ \nabla_{\mathbf{W}} \mathcal{L} &= \{\nabla_{w_{bk}} \mathcal{L}\}_{b=1, k=1}^{B, m} \in \mathbb{R}^{B \times m}, \\ \mathbf{V} &= \{v_k(\mathbf{x}_b)\}_{b=1, k=1}^{B, m} \in \mathbb{R}^{B \times m}, \\ \mathbf{c} &= (c_1, \dots, c_m)^\top \in \mathbb{R}^m, \quad \text{where } c_k := \cosh(2\sqrt{c} r_k) \sqrt{c}, \\ \mathbf{Z}_{\text{norm}} &\in \mathbb{R}^{m \times n} \quad \text{with rows} \quad \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|}. \end{aligned}$$

**Gradient with respect to weights  $\mathbf{Z}$ .** For arbitrary indices  $j, k$ :

$$\frac{\partial \mathcal{L}}{\partial z_{jk}} = \sum_b \nabla_{\tilde{y}_{bk}} \mathcal{L} \frac{\partial \tilde{y}_{bk}}{\partial z_{jk}} = \sum_b \nabla_{y_{bk}} \mathcal{L} \frac{\partial y_{bk}}{\partial z_{jk}}. \quad (19)$$

Using the chain rule through  $\mathbf{y}$ :

$$\frac{\partial y_{bk}}{\partial z_{jk}} = \frac{1}{d_b} \left( 1 - \frac{c w_{bk}^2}{d_b (d_b - 1)} \right) \frac{\partial w_{bk}}{\partial z_{jk}}, \quad \frac{\partial w_{bk}}{\partial z_{jk}} = \cosh(\sqrt{c} v_k(\mathbf{x}_b)) \frac{\partial v_k(\mathbf{x}_b)}{\partial z_{jk}}. \quad (20)$$

The gradient of  $v_k$  with respect to  $\mathbf{z}_k$  has two components:

$$\frac{\partial v_k(\mathbf{x}_b)}{\partial z_{jk}} = \frac{2}{\sqrt{c}} \frac{1}{\sqrt{1 + a_{bk}^2}} \left( \frac{\partial}{\partial z_{jk}} \left[ \frac{a_{bk}}{\|\mathbf{z}_k\|} \right] \right) \quad (21)$$

$$= \frac{2}{\sqrt{c}} \frac{1}{\sqrt{1 + a_{bk}^2}} \left( \frac{1}{\|\mathbf{z}_k\|} \frac{\partial a_{bk}}{\partial z_{jk}} - \frac{a_{bk} z_{jk}}{\|\mathbf{z}_k\|^3} \right), \quad (22)$$

where

$$\frac{\partial a_{bk}}{\partial z_{jk}} = \lambda_b \cosh(2\sqrt{c} r_k) \sqrt{c} \left( \frac{x_{bj}}{\|\mathbf{z}_k\|} - \frac{\langle \mathbf{x}_b, \mathbf{z}_k \rangle z_{jk}}{\|\mathbf{z}_k\|^3} \right) \quad (23)$$

Collecting all terms, we obtain a final expression for the derivative w.r.t. weight  $z_{jk}$ :

$$\frac{\partial \mathcal{L}}{\partial z_{jk}} = \sum_b \nabla_{y_{bk}} \mathcal{L} \frac{\cosh(\sqrt{c} v_k(\mathbf{x}_b))}{d_b} \left( 1 - \frac{c w_{bk}^2}{d_b(d_b - 1)} \right) \frac{2}{\sqrt{c(1 + a_{bk}^2)} \|\mathbf{z}_k\|} \quad (24)$$

$$\times \left[ \lambda_b \cosh(2\sqrt{c} r_k) \left( \frac{x_{bj}}{\|\mathbf{z}_k\|} - \frac{\langle \mathbf{x}_b, \mathbf{z}_k \rangle z_{jk}}{\|\mathbf{z}_k\|^3} \right) - \frac{a_{bk} z_{jk}}{\|\mathbf{z}_k\|^2} \right]. \quad (25)$$

In vectorized form:

$$\nabla_{\mathbf{Z}} \mathcal{L} = \mathbf{X}^\top \mathbf{S}^{(z)} - \mathbf{Z} \text{diag}(\mathbf{t}^{(z)}), \quad (26)$$

where we define:

$$S_{bk}^{(z)} = \nabla_{y_{bk}} \mathcal{L} \frac{\cosh(\sqrt{c} v_k(\mathbf{x}_b))}{d_b} \left( 1 - \frac{c w_{bk}^2}{d_b(d_b - 1)} \right) \frac{2\lambda_b \cosh(2\sqrt{c} r_k)}{\sqrt{c(1 + a_{bk}^2)} \|z_k\|^2}, \quad (27)$$

$$t_k^{(z)} = \sum_b \nabla_{y_{bk}} \mathcal{L} \frac{\cosh(\sqrt{c} v_k(\mathbf{x}_b))}{d_b} \left( 1 - \frac{c w_{bk}^2}{d_b(d_b - 1)} \right) \frac{2}{\sqrt{c(1 + a_{bk}^2)}} \quad (28)$$

$$\times \left( \frac{\lambda_b \cosh(2\sqrt{c} r_k) \langle \mathbf{x}_b, z_k \rangle}{\|z_k\|^4} + \frac{a_{bk}}{\|z_k\|^3} \right) \quad (29)$$

**Gradient with respect to biases  $\mathbf{r}$ .** For arbitrary index  $k$ :

$$\frac{\partial \mathcal{L}}{\partial r_k} = \sum_b \nabla_{\tilde{y}_{bk}} \mathcal{L} \frac{\partial \tilde{y}_{bk}}{\partial r_k} = \sum_b \nabla_{y_{bk}} \mathcal{L} \frac{\partial y_{bk}}{\partial r_k}. \quad (30)$$

Using the chain rule:

$$\frac{\partial y_{bk}}{\partial r_k} = \frac{1}{d_b} \left( 1 - \frac{c w_{bk}^2}{d_b(d_b - 1)} \right) \frac{\partial w_{bk}}{\partial r_k}, \quad \frac{\partial w_{bk}}{\partial r_k} = \cosh(\sqrt{c} v_k(\mathbf{x}_b)) \frac{\partial v_k(\mathbf{x}_b)}{\partial r_k}, \quad (31)$$

and

$$\frac{\partial v_k(\mathbf{x}_b)}{\partial r_k} = \frac{2}{\sqrt{c} \|\mathbf{z}_k\|} \frac{1}{\sqrt{1 + a_{bk}^2}} \frac{\partial a_{bk}}{\partial r_k}, \quad (32)$$

where

$$\frac{\partial a_{bk}}{\partial r_k} = 2\sqrt{c} \left( \lambda_b \langle \sqrt{c} \mathbf{x}_b, \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|} \rangle \sinh(2\sqrt{c} r_k) - (\lambda_b - 1) \cosh(2\sqrt{c} r_k) \right). \quad (33)$$

We get the following final expression:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial r_k} = & \sum_b \nabla_{y_{bk}} \mathcal{L} \frac{1}{d_b} \left( 1 - \frac{c w_{bk}^2}{d_b(d_b - 1)} \right) \cosh(\sqrt{c} v_k(\mathbf{x}_b)) \frac{4}{\|\mathbf{z}_k\|} \frac{1}{\sqrt{1 + a_{bk}^2}} \\ & \times \left( \lambda_b \langle \sqrt{c} \mathbf{x}_b, \frac{\mathbf{z}_k}{\|\mathbf{z}_k\|} \rangle \sinh(2\sqrt{c} r_k) - (\lambda_b - 1) \cosh(2\sqrt{c} r_k) \right), \end{aligned} \quad (34)$$

which can be expressed in vectorized form:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{r}} = & \sum_b \nabla_{\mathbf{y}_b} \mathcal{L} \odot \frac{1}{d_b} \left( \mathbf{1} - \frac{c \mathbf{w}_b^2}{d_b(d_b - 1)} \right) \odot \cosh(\sqrt{c} \mathbf{v}(\mathbf{x}_b)) \odot \frac{4}{\|\mathbf{Z}\|} \frac{1}{\sqrt{1 + \mathbf{a}_b^2}} \\ & \odot \left( \lambda_b \left\langle \sqrt{c} \mathbf{x}_b, \frac{\mathbf{Z}}{\|\mathbf{Z}\|} \right\rangle \odot \sinh(2\sqrt{c} \mathbf{r}) - (\lambda_b - 1) \odot \cosh(2\sqrt{c} \mathbf{r}) \right). \end{aligned} \quad (35)$$

### 3.4 Backward Pass Kernel

The backward pass kernel follows a hybrid design similar to the forward pass, leveraging optimized PyTorch functions for matrix multiplications while implementing the remaining gradient computations in a single, batched-parallelized Triton kernel.

As derived in Section 3.3, the gradients for  $\mathbf{X}$  and  $\mathbf{Z}$  can be expressed as a combination of matrix multiplications and diagonal scaling operations. Our implementation computes these gradients efficiently by calculating the intermediate matrices  $\mathbf{S}^{(x)}$ ,  $\mathbf{S}^{(z)}$  and vectors  $\mathbf{t}^{(x)}$ ,  $\mathbf{t}^{(z)}$  within the Triton kernel, followed by matrix operations in PyTorch.

The kernel computation proceeds in three sequential stages:

1. Compute  $\nabla_{\mathbf{Y}} \mathcal{L}$  as a Jacobian-vector product.
2. Compute  $\nabla_{\mathbf{W}} \mathcal{L}$  as a second Jacobian-vector product.
3. Compute the components  $\mathbf{S}^{(x)}$ ,  $\mathbf{S}^{(z)}$ ,  $\mathbf{t}^{(x)}$ , and  $\mathbf{t}^{(z)}$ .

The computation of  $\mathbf{S}^{(x)}$  and  $\mathbf{S}^{(z)}$  is fully parallelized across batch elements. The vector  $\mathbf{t}^{(x)}$  is accumulated independently per batch element, while  $\mathbf{t}^{(z)}$  and the bias gradients require atomic additions across the batch for reduction.

Upon completion of the kernel, the host code assembles the final gradients using PyTorch to perform the matrix multiplications defined by the derived expressions.

## 4 Experiments

In this section we run experiments to determine the efficiency of our implementation against HypLL, and an Euclidean neural network baseline. We then perform ablations to determine the

Experiments use FP32 precision and run on an NVIDIA RTX 2070 GPU with 8 GB of VRAM. Each experiment measures wall-clock time for a single training iteration to compute throughput (TFLOP/s) and monitors GPU memory allocation to evaluate memory efficiency.

Experiments code is released separately and can be accessed at [github.com/tiberuiiancu/hyp-ll-benchmark](https://github.com/tiberuiiancu/hyp-ll-benchmark).

### 4.1 HypLL vs. Fused FC

We evaluate both throughput and memory usage against two baselines: the original HypLL hyperbolic fully connected (FC) layer implementation and an Euclidean MLP of comparable size. The primary experimental setup uses a batch size of 256, a 3-layer architecture, and input and hidden dimensions of 4096.



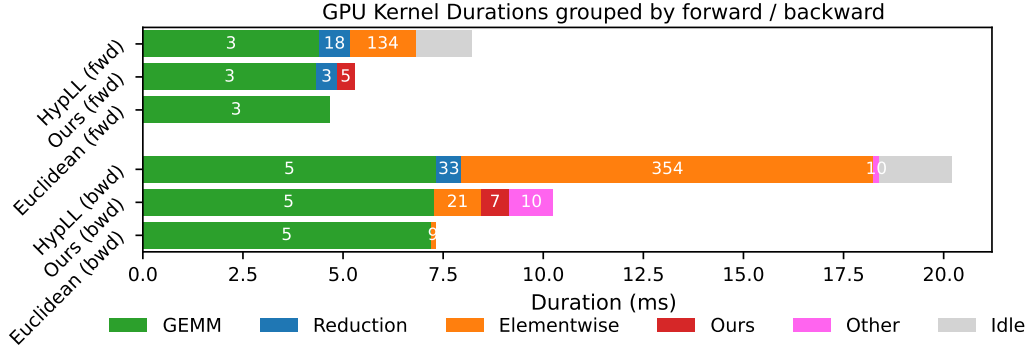


Figure 2: Comparison of the HypLL FC implementation against our optimized implementation on a single training iteration of a 3-layer MLP with ReLU activation on every layer except the last, grouped by forward and backward pass. The length of each bar indicates the execution duration in milliseconds, while the number within each segment indicates the number of GPU kernels executed. ‘GEMM’ is matrix multiplication. ‘Other’ represents non-kernel GPU operations, such as memory copies. ‘Idle’ is time spent waiting for kernels to be queued.

Figure 2 compares the original HypLL fully connected layer, our optimized fused implementation, and an Euclidean MLP baseline. The fused Triton implementation substantially reduces kernel launch overhead by combining element-wise operations and reductions into a single kernel. Each layer executes the forward pass using only four kernels (three for the final layer, as no activation is applied), minimizing GPU idle time. Consequently, execution latency decreases by over 33% relative to the HypLL implementation. Performance is within 10% of the Euclidean baseline, a reasonable trade-off given the additional computations required by the hyperbolic layer.

The backward pass, which is more computationally intensive, benefits even more from kernel fusion, achieving nearly a 50% reduction in latency. Compared to the Euclidean baseline, the backward pass remains approximately 33% slower. This is due not only to the heavier computation chain but also to costly memory copies caused by matrix transpositions. This overhead could likely be mitigated by storing weights in transposed form.

We evaluate whether fusion improves memory efficiency. Table 1 compares the HypLL implementation, our fused approach, and the Euclidean baseline. Our implementation reduces the memory required for activations—the intermediate results used during the backward pass—by roughly half; however, it consumes approximately three times more memory than the Euclidean counterpart. This increase is due to the compute-intensive backward pass of the Poincaré fully connected layer, which requires storing forward-pass intermediates that would otherwise need to be recomputed. Our design principle is to cache expensive operations, such as matrix multiplications or reductions, while recomputing inexpensive element-wise operations.

Despite these savings, the overall peak memory shows only modest reduction. This is primarily because optimizer states and model parameters dominate the memory footprint. Moreover, peak memory is measured during the optimizer step after the backward pass, by which point activations have already been released. These results suggest that the current network is too shallow for activation memory usage to significantly impact peak consumption. We anticipate that in deeper or more complex architectures, such as ResNets [6] or Transformers [15], activations will constitute a more substantial memory bottleneck.

|             | HypLL   | Ours    | Euclidean |
|-------------|---------|---------|-----------|
| Activations | 190 MB  | 100 MB  | 35 MB     |
| Peak        | 1.13 GB | 1.08 GB | 1 GB      |

Table 1: Memory usage by MLP implementation on the same 3-layer architecture. Activations represent intermediate outputs produced by layers during the forward pass.

Parameter sweep FLOPS comparison

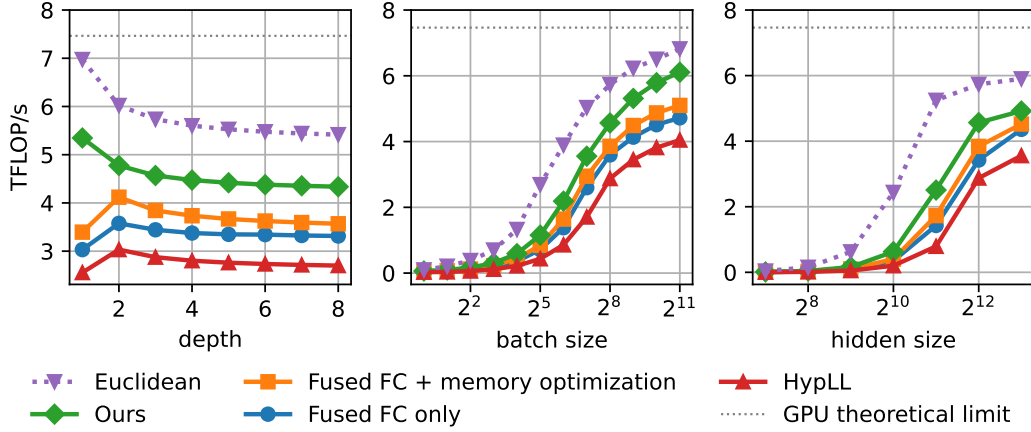


Figure 3: TFLOP/s measured for a single training iteration of a 3-layer MLP under various ablations. Training hyperparameters (batch size, input size, hidden size, depth) are held constant while one parameter is varied at a time. In the hidden size sweep, both input and hidden dimensions are scaled jointly. The dotted line indicates the theoretical FP32 throughput limit of the RTX 2070.

From this experiment we conclude that kernel fusion and targeted caching effectively reduce activation memory and accelerate both forward and backward passes, with peak memory still dominated by optimizer states.

## 4.2 Ablations

To isolate the contribution of each optimization, we perform three ablation experiments:

- Full fusion — the complete Triton implementation where the FC, activation, and associated hyperbolic operations are fused into a single kernel;
- Without activation fusion — only the FC component is fused, with the activation handled separately;
- FC fusion + memory optimization — the FC kernel is fused and we apply the memory-management optimization that minimizes redundant GPU allocations (e.g., avoiding zero-initialized allocations and extra device-to-device copies); the activation function remains unfused.

We independently sweep batch size, network depth, and hidden size (the latter jointly with input size) while keeping the remaining parameters fixed. Each run measures the time for a single training iteration, from which we derive floating-point throughput (TFLOP/s) based on estimated operation counts as a function of architecture and hyperparameters (details in Appendix B). TFLOP/s, a standard GPU efficiency metric, reflects how close an implementation operates to the device’s theoretical arithmetic limit. The results are shown in Figure 3.

Across all sweeps, performance improves consistently with each optimization stage, while the Euclidean MLP maintains the highest throughput and the original HypLL implementation remains the lowest. As network depth increases, throughput decreases—a result of accumulating CPU overhead (e.g., data movement, synchronization, kernel launches) and non-accounted operations such as loss computation.

Larger batch sizes lead to higher arithmetic intensity and better hardware utilization. With a batch size of 2048, our fused kernel achieves up to 80% of the GPU’s theoretical TFLOP/s and comes within 10% of the Euclidean model. This is expected, as larger batches shift the workload toward matrix multiplications, where our optimizations are most effective. When varying the hidden size, we observe similar trends—performance scales predictably across ablations, but the Euclidean model

maintains an advantage. This difference stems from our kernel launch configuration of shape  $(B, )$ , which limits parallelism but enables efficient in-kernel reductions. Increasing grid dimensionality would improve parallelism but require more kernel launches, reintroducing synchronization overhead and reducing overall efficiency. This reflects an inherent trade-off imposed by hyperbolic space operations.

## 5 Discussion

In this section we address some of the limitations of the work, and discuss possible directions for future research.

**Training convergence.** To verify that our fused implementation maintains correct training behavior, we trained a hyperbolic ResNet on CIFAR-10 [7] for 5 epochs, comparing against the original HypLL implementation using identical weight initialization. Our model’s loss remained approximately 5% higher. Due to time constraints, we did not further investigate this discrepancy, but we suspect it arises from floating-point precision (FP32) effects. Specifically, our implementation’s gradients appear to have slightly smaller magnitudes, leading to slower convergence. A modest increase in learning rate could likely compensate for this effect.

**Convolutions.** HypLL implements convolutions as an unfold–FC–fold sequence, where the FC layer performs a batched matrix multiplication over 3D tensors. Our FC implementation does not directly support this 3D batched multiplication; instead, we flatten and reorder the input to fit a 2D matrix multiplication. While this produces correct results, it introduces additional memory operations that degrade performance. A natural next step is to design a dedicated GPU kernel for hyperbolic convolutions that operates directly on 4D tensors, eliminating unnecessary data movement and improving efficiency.

**PyTorch Compile.** PyTorch’s `torch.compile` can accelerate operations by analyzing the computational graph and generating optimized Triton kernels, essentially automating the fusion and caching strategies explored in this work. However, it has several limitations that reduce its suitability for large-scale experiments. Compilation is significantly slower than direct Triton compilation and must be performed on every machine executing the code. In our experiments, larger models sometimes failed to compile due to out-of-memory errors or other difficult-to-diagnose issues. Additionally, while `torch.compile` provides notable speedups, it did not surpass the performance of our optimized kernels and scaled less efficiently with increasing model size. For these reasons, we excluded it from our experimental evaluation.

**Beyond MLPs.** Looking beyond the fully connected layers explored in this work, we see two key directions for scaling hyperbolic deep learning: first, extending our fusion and caching strategies to complex modern architectures like Transformers; and second, developing a standardized, high-performance backend of optimized hyperbolic kernels—an analog to cuBLAS/cuDNN. A unified library of this kind would provide a common computational foundation for various frameworks, accelerating both efficiency and broader adoption.

## 6 Conclusion

In this paper we introduced a fused GPU implementation of the Poincaré fully connected layer that substantially improves the efficiency of hyperbolic neural networks. By decomposing the layer into atomic operations and fusing element-wise maps and reductions in Triton, we reduced kernel launch overhead and global memory traffic, achieving up to  $2\times$  faster training and nearly halving activation memory usage compared to HypLL.

These results demonstrate that geometry-aware kernel fusion can close much of the performance gap between hyperbolic and Euclidean networks without altering their mathematical foundations. While our work focused on the fully connected layer, the same principles extend naturally to hyperbolic convolutions and attention mechanisms. We see this as an initial step toward a unified “hyperbolic cuBLAS/cuDNN” stack—an optimized backend enabling scalable, high-performance deep learning on curved spaces.

## References

- [1] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR) 2015*, 2015. URL <https://arxiv.org/abs/1409.0473>.
- [2] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*, 2014.
- [3] NVIDIA Corporation. NVIDIA cuBLAS. <https://developer.nvidia.com/cublas>, 2025. Accessed: 2025-11-25.
- [4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)35*, pages 16344–16359, 2022. URL <https://proceedings.neurips.cc/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html>.
- [5] Octavian-Eugen Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In *Advances in Neural Information Processing Systems31*, pages 5350–5360. Curran Associates, Inc., 2018. URL <http://papers.nips.cc/paper/7780-hyperbolic-neural-networks.pdf>.
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. URL <https://arxiv.org/abs/1512.03385>.
- [7] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [8] Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems 30*, pages 6338–6347. Curran Associates, Inc., 2017. URL <https://papers.nips.cc/paper/7213-poincare-embeddings-for-learning-hierarchical-representations>.
- [9] NVIDIA Corporation. *CUDA: Compute Unified Device Architecture*, 2025. URL <https://developer.nvidia.com/cuda-toolkit>.
- [10] OpenAI. Triton: An open source deep learning compiler for ai accelerators. <https://triton-lang.org/>, 2023.
- [11] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- [12] Ryohei Shimizu, Yusuke Mukuta, and Tatsuya Harada. Hyperbolic neural networks++. In *International Conference on Learning Representations (ICLR)2021*, 2021. URL <https://openreview.net/forum?id=Ec85b0tUwBA>. Poster.
- [13] Max van Spengler, Erwin Berkhout, and Pascal Mettes. Poincaré resnet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5396–5405, 2023. doi: 10.1109/ICCV51070.2023.00499. URL <https://arxiv.org/abs/2303.14027>.
- [14] Max van Spengler, Philipp Wirth, and Pascal Mettes. Hyppl: The hyperbolic learning library. In *Proceedings of the 31st ACM International Conference on Multimedia (MM '23)*, pages 9676–9679. ACM, 2023. doi: 10.1145/3581783.3613462. URL [https://github.com/maxvanspengler/hyperbolic\\_learning\\_library](https://github.com/maxvanspengler/hyperbolic_learning_library).

- [15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS) 30*, pages 5998–6008, 2017. URL <https://arxiv.org/abs/1706.03762>.

## A Triton Kernel Design Discussion

A key design consideration in our Triton kernel implementation was the scope of parallelization. While our kernels are inherently parallel across the batch dimension, we also evaluated tiling across the input columns to enhance scalability for very large layers. However, this column-wise tiling necessitates multiple kernel launches—for instance, the backward pass would require three separate triton kernels, essentially resulting in three separate layers.

This fragmentation negates the benefits of kernel fusion, particularly for smaller input sizes where kernel launch overhead is most pronounced. We empirically compared both designs and found that the batch-only tiling strategy was faster for inputs smaller than 1024 elements and performed comparably for sizes up to 16384. Since projection layers in practical models rarely exceed this scale, we opted for the batch-only design to minimize launch overhead and maintain kernel fusion.

## B FLOP Accounting Methodology

To compare our custom hyperbolic kernels to a Euclidean baseline, we estimated FLOPs analytically for a single forward–backward layer pass and converted the measured wall-clock time (in ms) into TFLOP/s. We treat each layer as operating on a batch of size  $b$ . The first layer maps  $k \rightarrow m$ ; the remaining  $d - 1$  layers map  $m \rightarrow m$ . Total FLOPs are simply  $d$  times the per-layer count (with the first layer using  $k$ , others using  $m$ ).

Given a runtime of  $\text{ms}$  milliseconds,

$$\text{TFLOP/s} = \frac{\text{FLOP}_{\text{total}}}{10^{12}} \cdot \frac{1000}{\text{ms}}.$$

Our FLOP counts come directly from the operations implemented in the fused CUDA kernels: matrix multiplications, pointwise ops, vector reductions, and the hyperbolic log/exp maps. Every multiply or add is counted as one FLOP; fused kernels are decomposed into their algebraic components. These expressions match the exact work performed by the kernels used in the benchmarks.

**Hyperbolic Layer.** A hyperbolic layer consists of a fully-connected block and the logmap–ReLU–expmap activation pipeline, both in forward and backward mode. Collecting terms:

$$\begin{aligned} \text{FLOP}_{\text{shyp}}(b, k, m) = & \underbrace{b(2k + 29m + 12)}_{\text{Triton kernel}} + \underbrace{2bmk}_{\text{GEMM}} + \underbrace{2km}_{\text{norm}} + \underbrace{(4bm + bm + 7bm)}_{\text{activation}} \quad (\text{forward pass}) \\ & + \underbrace{84bm}_{\text{Triton kernel}} + 2 \times \underbrace{2bmk}_{\text{GEMM}} + \underbrace{(2bk + 2km)}_{\text{vector ops}} + \underbrace{(9bm + 5bm)}_{\text{activation}} \quad (\text{backward pass}) \end{aligned}$$

**Euclidean Layer.** The Euclidean baseline layer is a standard linear layer plus ReLU, giving:

$$\text{FLOP}_{\text{seuc}}(b, k, m) = 6bkm + 2bm.$$

The total for a network of depth  $d$  is obtained by applying the single-layer formulas to the  $(k \rightarrow m)$  input layer and the  $(m \rightarrow m)$  hidden layers:

$$\text{FLOP}_{\text{total}} = \text{FLOP}_{\text{slayer}}(b, k, m) + (d - 1) \text{FLOP}_{\text{slayer}}(b, m, m).$$